EfficientNet-B0: Official Hyperparameters and What You Can Use

Read from the tensorflow/tpu EfficientNet code (models/official/efficientnet). The tag in square brackets shows which file each value comes from.

1. Parameters: official vs. what you can implement

Parameter	Official repo	What you can implement (PyTorch, RTX 4050)
Model	EfficientNet-B0: width 1.0, depth 1.0, 7 MBConv stages, squeeze-excitation 0.25, Swish/SiLU [efficientnet_builder.py]	torchvision efficientnet_b0(weights=None, num_classes=2)
Initialization	Trained from scratch on ImageNet; checkpoints released [README]	Random initialization, no pretrained weights
Optimizer	RMSProp: decay 0.9, momentum 0.9, epsilon 0.001 [utils.py, build_optimizer defaults]	timm.optim.RMSpropTF(alpha=0.9, momentum=0.9, eps=1e-3). Use this instead of torch.optim.RMSprop (different epsilon placement and initial state)
Base learning rate	0.016, defined as the rate 'when train batch size is 256' [main.py]	0.016 (the base value)
LR scaling	scaled_lr = base_learning_rate x (train_batch_size / 256) [main.py]. 0.128 at batch 2048	0.016 x 4 / 256 = 0.00025 at batch 4
Warmup	5 epochs, linear from 0 up to scaled_lr [utils.py]	5 epochs, linear, updated every step
LR decay	Exponential, x0.97 every 2.4 epochs, staircase [utils.py]	Same: x0.97 every 2.4 epochs, staircase
Weight decay	1e-5, added to the loss as weight_decay x l2_loss(v) for every trainable variable whose name does not contain 'batch_normalization' (so biases ARE decayed) [main.py]	1e-5 on conv/linear weights and biases; no decay on BatchNorm weights/biases
Label smoothing	0.1 on softmax cross-entropy [main.py]	Not applicable to BCE. Leave off (or use smoothed targets as a documented project choice)
Loss	Softmax cross-entropy over 1000 classes [main.py]	BCEWithLogitsLoss on 2 outputs
EMA of weights	Decay 0.9999 with num_updates = global_step (decay ramps up early); averages trainable variables and BatchNorm moving mean/variance; evaluation uses the EMA weights [main.py, utils.py]	EMA 0.9999 with ramp min(0.9999, (1+t)/(10+t)); also average BatchNorm running mean/variance; validate and test with the EMA model
Dropout	0.2 for B0 (final layer) [efficientnet_builder.py, paper]	0.2 (torchvision default)
Stochastic depth	Survival probability 0.8 [efficientnet_builder.py, paper]	stochastic_depth_prob = 0.2
BatchNorm	Momentum 0.99, epsilon 1e-3 [efficientnet_builder.py, paper]	momentum = 0.01 (PyTorch convention), eps = 1e-3
Train batch size	2048 [main.py]	4 (hardware limit)
Eval batch size	1024 [main.py]	4 to 8
Training length	218,949 steps, about 350 epochs at batch 2048, on 1,281,167 images [main.py]	70 epochs
Steps per epoch	num_train_images / train_batch_size = 625.57 [main.py]	ceil(train size / 4)
Evaluation schedule	Every 6,255 steps, about 10 epochs [main.py]	Every epoch
Input size	224 x 224 for B0 [preprocessing.py]	512 x 512
Normalization	Subtract MEAN_RGB = [0.485, 0.456, 0.406] x 255, divide STDDEV_RGB = [0.229, 0.224, 0.225] x 255 [main.py]	Normalize with mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225] on [0, 1] images (same thing)

Parameter	Official repo	What you can implement (PyTorch, RTX 4050)
Precision	float32 by default; bfloat16 optional on TPU (use_bfloat16 = False) [main.py]	CUDA FP16 autocast + GradScaler
Hardware	Cloud TPU, TensorFlow 1.13 or later [README]	Single RTX 4050 6 GB, PyTorch

2. Learning rate in detail

- **Optimizer:** RMSProp with decay 0.9, momentum 0.9, epsilon 0.001 (utils.py build_optimizer). The momentum flag in main.py (0.9) is not used directly.
- **Base rate and scaling:** base_learning_rate = 0.016 'when train batch size is 256'. main.py computes `scaled_lr = base_learning_rate * (train_batch_size / 256.0)`. At the default batch of 2048 this is 0.128.
- **Warmup (utils.py build_learning_rate):** warmup_epochs = 5, warmup_steps = int(5 x steps_per_epoch). While global_step < warmup_steps the rate is `scaled_lr * global_step / warmup_steps` (linear, starting from 0).
- **Decay:** tf.train.exponential_decay(scaled_lr, global_step, decay_steps, 0.97, staircase=True) with decay_steps = steps_per_epoch x 2.4. The count of decays starts at step 0, not at the end of warmup, so when warmup ends the schedule switches to the already-decayed value (0.128 x 0.97^2 = 0.1204 at step 3,127).
- **Paper:** Section 5.2 states 'initial learning rate 0.256 that decays by 0.97 every 2.4 epochs' with RMSProp decay 0.9 and momentum 0.9. The paper does not state the batch size, and 0.256 is twice the code's 0.128 at batch 2048. For an 'official repo' claim, follow the code values.
- **Other schedule facts:** steps_per_epoch is a float division (625.57). Evaluation uses the EMA weights.

	Official (batch 2048)	Yours (batch 4, 805 steps per epoch if train = 3,219 images)
Peak LR	0.016 x 2048 / 256 = 0.128	0.016 x 4 / 256 = 0.00025
Warmup	5 epochs = 3,127 steps	5 epochs = 4,025 steps
Decay interval	2.4 epochs = about 1,501 steps	2.4 epochs = 1,932 steps
LR at the end of warmup	0.128 x 0.97^2 = 0.1204	0.00025 x 0.97^2 = 0.000235
LR at the end of training	0.128 x 0.97^145 = about 0.00155 (350 epochs)	0.00025 x 0.97^29 = about 0.000103 (70 epochs)

Note: using 0.016 unscaled at batch 4 is not what the official code does. It would be 64 times the scaled value.

3. Augmentation and preprocessing in detail

Are augmentations enabled? Basic augmentation is always on in training. AutoAugment and RandAugment are off by default and are switched on with a flag.

Item	Official repo	Default
Random crop (train)	Random 'Inception-style' crop: area range (0.08, 1.0), aspect ratio (3/4, 4/3), min_object_covered 0.1, max_attempts 10; if the crop fails it falls back to a center crop [preprocessing.py]	On
Resize	Bicubic resize to 224 x 224 (both train and eval) [preprocessing.py]	On
Horizontal flip	random_flip_left_right [preprocessing.py]	On
AutoAugment	augment_name='autoaugment' calls distort_image_with_autoaugment(image, 'v0') (ImageNet policy v0). Enabled with --augment_name=autoaugment [preprocessing.py, README]	Off (augment_name = None)
RandAugment	augment_name='randaugment' with randaug_num_layers and randaug_magnitude. README: B5 uses 2 layers, magnitude 17; B7 and B8 use 2 layers, magnitude 28. No B0 values given [preprocessing.py, README]	Off
Mixup / CutMix	Not present in the files read	Not used
Eval preprocessing	Center crop of size image_size / (image_size + 32) x min(height, width), i.e. 0.875 of the short side, then bicubic resize to 224; no flip [preprocessing.py]	n/a

- **Accuracy with and without AutoAugment (README, B0 top-1):** baseline preprocessing 76.7%, AutoAugment 77.1%. The paper's 77.1% B0 result (Table 2) corresponds to the AutoAugment setting, and the paper lists AutoAugment in its training details.
- **What you can implement:** (a) official-equivalent: RandomResizedCrop(512, scale=(0.08, 1.0), ratio=(3/4, 4/3), interpolation=BICUBIC) + RandomHorizontalFlip, no AutoAugment. torchvision has no min_object_covered, and a 0.08 crop can cut out small lesions on an X-ray. (b) X-ray-safe, project choice: Resize 544, RandomCrop 512, RandomHorizontalFlip (current script). Report (b) as a documented deviation.
- **AutoAugment/RandAugment for TBX11K:** they use colour operations such as Invert, Solarize and Posterize that are designed for natural images. I would leave them off and state that the official default is also off.
- **Eval:** resize to 512 (images are already 512 x 512); the official 0.875 center crop is optional.

4. What could not be confirmed

- **Where each file was read:** the official GitHub pages for main.py and utils.py could not be opened in my environment. main.py came from a replica of the same folder (an older snapshot with no augment_name flag), and preprocessing.py, utils.py and efficientnet_builder.py from a Hugging Face mirror of the original files. The README was read from the official repo. Values should be re-checked against the live tensorflow/tpu files before you claim an exact match.
- **AutoAugment details:** the 'v0' policy is named in preprocessing.py, but I could not open EfficientNet's autoaugment.py itself, so the list of operations is not reproduced here.
- **Not checked:** stochastic-depth scaling across blocks, weight initialization, and the exact order of crop, flip and AutoAugment inside preprocess_for_train.
- **Newer official main.py:** it may define more flags (for example augment_name or mixup) than the older snapshot I read.

Sources

- github.com/tensorflow/tpu/tree/master/models/official/efficientnet - README (official)
- main.py: github.com/rajneeshaggarwal/google-efficientnet (replica of the folder)
- utils.py, preprocessing.py, efficientnet_builder.py: huggingface.co/spaces/Aryan6192/deep (external_data/original_tf, mirror)
- Tan and Le, EfficientNet: Rethinking Model Scaling for CNNs, arXiv 1905.11946, Section 5.2
- timm RMSpropTF: github.com/huggingface/pytorch-image-models (timm/optim/rmsprop_tf.py)